

Aula 5 - Tratamento de Exceções e Logging em Java

Duração: 2 horas

Objetivos:

- Criar exceções customizadas para domínio específico
- Implementar logging eficiente com Log4j/SLF4J
- Integrar monitoramento de erros com Sentry

Parte Teórica (40 min)

1. Hierarquia de Exceções em Java

Diagram

Code

```
classDiagram
    Throwable <|-- Exception
    Throwable <|-- Error
    Exception <|-- RuntimeException
    Exception <|-- CustomException
```

- **Checked vs Unchecked:**
 - Checked (herdam de `Exception`): Obrigatório tratar (ex: `IOException`)
 - Unchecked (herdam de `RuntimeException`): Não exigem try-catch (ex: `NullPointerException`)

2. Boas Práticas em Exceções

- **Custom Exceptions:** Criar para erros de negócio (ex: `SaldoInsuficienteException`)
- **Evite:**
 - Logar e relançar (`catch-log-throw`)
 - `Exception` genérica (`catch(Exception e)`)
- **Use:**
 - Try-with-resources para recursos (`try (FileReader fr = new ...)`)
 - Mensagens claras (`throw new UsuarioNaoEncontradoException("ID: " + id)`)

3. Logging Eficiente

Framework	Vantagem	Quando Usar
Log4j2	Alta performance	Aplicações críticas
SLF4J	Facilidade de troca de implementação	Quando precisa de flexibilidade

Níveis de Log:

- `DEBUG > INFO > WARN > ERROR > FATAL`

4. Monitoramento (Sentry/ELK)

- **Sentry:** Erros em tempo real com stacktrace
- **ELK Stack** (Elasticsearch + Logstash + Kibana): Análise histórica de logs

Parte Prática (1h20min)

Atividade 1: Exceções Customizadas

Cenário: Sistema bancário com:

- `SaldoInsuficienteException`
- `LimiteSaqueExcedidoException`

Passo a passo:

1. Crie a exceção customizada:

java

```
public class SaldoInsuficienteException extends RuntimeException {  
    public SaldoInsuficienteException(double saldoAtual) {  
        super("Saldo insuficiente. Disponível: " + saldoAtual);  
    }  
}
```

2. Use no serviço:

java

```
class ContaService {  
    void sacar(double valor) {  
        if (valor > saldo) {
```

```

        throw new SaldoInsuficienteException(saldo);
    }
    // ...
}
}

```

Exercício: Crie `LimiteSaqueExcedidoException` e aplique regra de máximo R\$ 1000 por saque.

Atividade 2: Configurando Log4j2

1. Adicione ao `pom.xml` :

xml

```

<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>2.20.0</version>
</dependency>

```

2. Crie `log4j2.xml` :

xml

```

<Configuration>
    <Appenders>
        <Console name="Console" target="SYSTEM_OUT">
            <PatternLayout pattern="%d{HH:mm:ss} [%t] %-5level %logger{36} - %msg%n"/>
        </Console>
    </Appenders>
    <Loggers>
        <Root level="DEBUG">
            <AppenderRef ref="Console"/>
        </Root>
    </Loggers>
</Configuration>

```

3. Use no código:

java

```

private static final Logger logger = LogManager.getLogger(ContaService.class);

void transferir() {
    logger.info("Iniciando transferência");
    try {

```

```

        // ...
    } catch (SaldoInsuficienteException e) {
        logger.error("Falha na transferência: {}", e.getMessage());
        throw e;
    }
}

```

Exercício: Adicione um `FileAppender` para salvar logs em arquivo.

Atividade 3: Integrando Sentry (Opcional)

1. Crie conta em sentry.io
2. Adicione dependência:

xml

```

<dependency>
  <groupId>io.sentry</groupId>
  <artifactId>sentry-log4j2</artifactId>
  <version>6.25.0</version>
</dependency>

```

3. Modifique `log4j2.xml` :

xml

```

<Appenders>
  <Sentry name="Sentry" level="ERROR"/>
</Appenders>

```

4. Teste captura de erro:

java

```

try {
    throw new RuntimeException("Erro crítico!");
} catch (Exception e) {
    logger.error("Erro capturado pelo Sentry", e);
}

```

Desafio Prático (30 min)

Contexto: Refatore o código abaixo aplicando:

1. Exceção customizada para idade inválida
2. Logging com SLF4J
3. Tratamento elegante com try-with-resources

java

```
class CadastroUsuario {  
    void cadastrar(String nome, int idade) {  
        if (idade < 18) {  
            System.err.println("Usuário menor de idade");  
        }  
        // ...  
    }  
  
    void salvarArquivo(String path) {  
        FileWriter fw = null;  
        try {  
            fw = new FileWriter(path);  
            // ...  
        } catch (IOException e) {  
            e.printStackTrace();  
        } finally {  
            if (fw != null) fw.close();  
        }  
    }  
}
```

Encerramento

- **Discussão:** Quando uma exceção deveria ser checked vs unchecked?
- **Próxima aula:** Testes Unitários com JUnit 5 (Aula 6)

Tarefa:

- Configurar Sentry em um projeto pessoal
- Extra: Explorar Kibana para visualização de logs

Recursos:

- [Log4j2 Configuration](#)

- [Sentry Java Docs](#)

Pronto para transformar seu tratamento de erros em algo profissional? Dúvidas? 😊🚀